Análise Estática Básica

A análise de malware é uma atividade inerentemente perigosa. Por sua natureza destrutiva, um código malicioso pode causar danos permanentes ao sistema operacional, sequestrar dados ou infectar a rede local em segundos. Por isso, antes de iniciar qualquer estudo, o primeiro passo não é abrir o arquivo, mas sim garantir o isolamento total.

O Laboratório de Análise: Máquinas Virtuais (VMs)

Para investigar ameaças sem comprometer o hardware físico, utilizamos ambientes virtualizados. A grande vantagem das VMs é a função de Snapshot:

- Snapshot: Captura o estado limpo da máquina com todas as ferramentas instaladas.
- Restauração: Após infectar a VM durante a análise, basta reverter para o snapshot anterior, eliminando qualquer vestígio do malware e retornando a um ambiente seguro e pronto para o próximo teste.

Plataformas de Referência

Não é necessário instalar ferramenta por ferramenta manualmente. A comunidade de segurança utiliza distribuições prontas que já agregam o que há de melhor para análise:

1. FLARE VM (Windows): Criada pela Mandiant, é a escolha padrão para análise de malware voltado ao ecossistema Windows. Por ser baseada em Windows, permite tanto a análise estática quanto a execução controlada (análise dinâmica).
2. REMnux (Linux): Uma distribuição focada em engenharia reversa. É excelente para analisar scripts, arquivos PDF maliciosos, extrair strings e realizar análises estáticas rápidas sem o risco de o malware de Windows se autoexecutar por acidente.

Dando continuidade ao nosso guia de Análise Estática Básica, o próximo passo fundamental após a preparação do ambiente é a extração de informações sem a execução do arquivo. A técnica mais comum e poderosa para isso é a Busca por Strings.

Extração de Strings: O Primeiro Contato com o Código

As Strings são sequências de caracteres legíveis (como palavras e frases) incorporadas dentro do código binário. Elas funcionam como "pistas" deixadas pelo autor do malware que revelam as capacidades e intenções do arquivo.

Como funciona a extração?

As ferramentas de análise percorrem o arquivo bit a bit em busca de sequências de caracteres (ASCII ou Unicode) que terminam com um caractere nulo.

> Atenção: Nem tudo o que parece texto é uma string útil. Muitas vezes, instruções de processador ou endereços de memória podem "parecer" texto

aleatório. O analista deve filtrar esse "lixo" para encontrar os verdadeiros Indicadores de Comprometimento (IoCs).

O que um analista deve procurar:

- APIs do Windows: Nomes como `CreateProcess`, `InternetOpen` ou `ShellExecute` indicam o que o malware tenta fazer no sistema.
- Rede: Endereços IP, domínios de comando e controle (C2) ou URLs de download.
- Artefatos Diversos: Endereços de carteiras de criptomoedas (em ransomwares), mensagens de erro personalizadas ou caminhos de diretórios (PDB paths).

Ferramentas Essenciais no Laboratório

No nosso laboratório (FLARE VM ou REMnux), utilizamos ferramentas específicas para essa tarefa:

1. `strings.exe` (Sysinternals): A ferramenta clássica de linha de comando. É rápida e permite exportar os resultados para um arquivo de texto para análise posterior.
    - *Comando:* `strings <caminho_do_arquivo>`
2. PEstudio: Uma das ferramentas gráficas mais completas. Além de listar as strings, ela as compara com uma "lista negra" de termos suspeitos e indica quais APIs são frequentemente usadas por malwares (usando referências como MalAPI).

O Desafio das Strings Ofuscadas

Autores de malware modernos sabem que strings revelam seus segredos. Por isso, eles costumam escondê-las (ofuscação). Se uma busca comum retornar pouca informação, o malware pode estar usando strings ocultas.

FLOSS (FireEye Labs Obfuscated String Solver)

Para combater a ofuscação, utilizamos o FLOSS. Ao contrário das ferramentas comuns, o FLOSS tenta emular a execução de partes do código para "desembrulhar" e revelar as strings que só apareceriam quando o malware estivesse rodando.

- Dica Prática: Se o `strings.exe` não mostrar quase nada, o `FLOSS` é sua próxima parada obrigatória.
    - *Comando para strings ocultas:* `floss --no-static-strings <caminho_do_arquivo>`

Qual dessas amostras contém a string 'DbgView.exe'?
Resposta: 6

```
FLARE Fri 04/03/2026 13:19:40.62
C:\Users\Administrator\Desktop\mal>floss --no-static-strings 6
WARNING:envi.codeflow:parseOpcode error at 0x1400596c8 (addCodeFlow(0x140058558)): InvalidInstruction("'c5eeffff440fb702
33c041baffff0000' at 0x1400596c8L",)
WARNING:envi.codeflow:parseOpcode error at 0x14004edc2 (addCodeFlow(0x14004b0d4)): InvalidInstruction("'db75178b1509f304
00488d0d2ca50300' at 0x14004edc2L",)
WARNING:envi.codeflow:parseOpcode error at 0x14004e645 (addCodeFlow(0x14004e5b0)): InvalidInstruction("'db75178b1586fa04
00488d0df9ab0300' at 0x14004e645L",)

FLOSS decoded 1 strings
@@AD

FLOSS extracted 45 stackstrings
Dbgview.exe
Dbgview.exe
AVQZAWI
Dbgview.exe
Dbgview.exe
Dbgview.exe
Dbgview.exe
Dbgview.exe
```

Hashes: A Identidade Digital do Malware

Na análise de malware, nomes de arquivos são irrelevantes, pois podem ser alterados facilmente. Para identificar uma amostra de forma única e imutável, utilizamos funções Hash.

1. Identificação Única (MD5, SHA-1, SHA-256)

Um hash é o resultado de um cálculo matemático sobre o conteúdo do arquivo. Se um único bit mudar, o hash muda completamente.

- SHA-256: É o padrão atual de segurança. É o que utilizamos para buscar amostras no *VirusTotal* ou *MalwareBazaar*.
- MD5/SHA-1: Embora ainda usados, são mais suscetíveis a colisões e menos seguros para identificação crítica.

Hashes de Similaridade: Encontrando "Parentes"

Diferente dos hashes comuns, existem tipos especiais que nos ajudam a agrupar malwares da mesma família ou que compartilham o mesmo código, mesmo que o SHA-256 seja diferente.

A. Imphash (Import Hash)

O Imphash é o hash da Tabela de Importação do arquivo (as DLLs e funções que ele chama e a ordem em que aparecem).

- Utilidade: Se dois malwares diferentes têm o mesmo Imphash, eles provavelmente foram compilados da mesma forma ou usam a mesma estrutura de ataque. No laboratório, o PEstudio é a ferramenta ideal para encontrar esse valor.

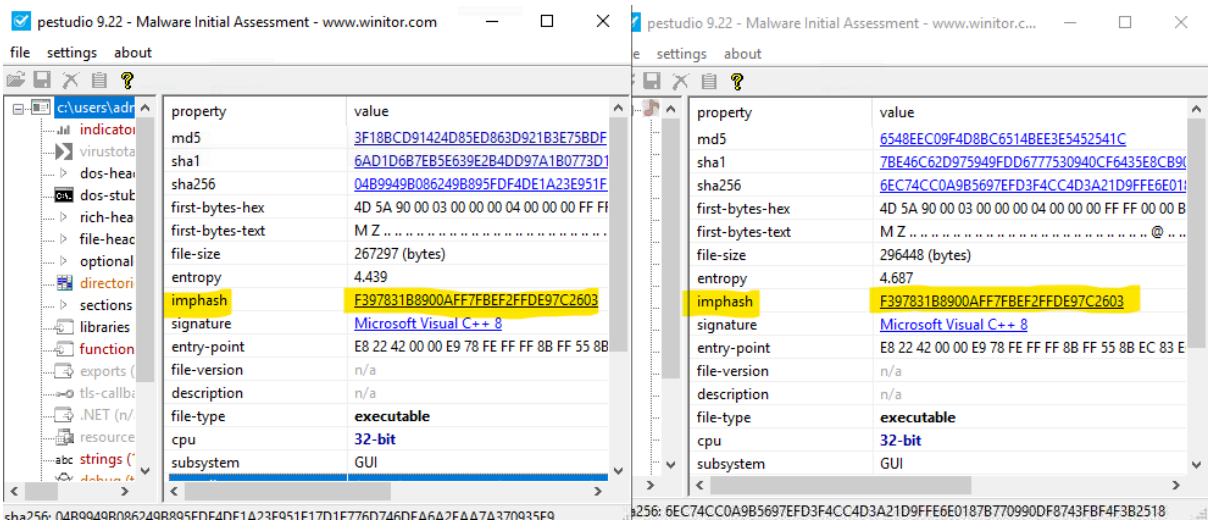
B. Fuzzy Hashing (ssdeep)

O ssdeep utiliza o *Context Triggered Piecewise Hashing* (CTPH). Ele divide o arquivo em pedaços e gera hashes para cada parte.

- Utilidade: Permite comparar dois arquivos e dizer, por exemplo, que eles são 99% iguais. Isso é vital quando um atacante faz uma pequena mudança no código apenas para burlar antivírus baseados em hash comum.

Exercício Prático: Comparação no Laboratório

Questão: Nas amostras do diretório `Desktop\mal\`, qual possui o mesmo imphash que o arquivo 3?
Resposta: 1



A esquerda amostra 3⬆️ A direita amostra 1⬆️

Utilizando o utilitário ssdeep, qual é a porcentagem de correspondência dos arquivos mencionados acima?
Resposta: 93

```
C:\Users\Administrator\Desktop>ssdeep-2.14.1\ssdeep.exe -l -r -d mal\*
mal\3 matches mal\1 (93)
```

Assinaturas e Capacidades: Entendendo o Comportamento

Identificar um malware pelo hash é apenas o começo. Para entender a real ameaça, precisamos de Assinaturas — padrões específicos (sequências de bytes ou textos) que denunciam comportamentos maliciosos conhecidos.

1. Regras YARA: O Padrão da Indústria

O YARA é a ferramenta essencial para correspondência de padrões. Ele permite que analistas criem regras baseadas em strings e valores hexadecimais para detectar famílias de malware ou técnicas específicas.

- Uso: Serve tanto para identificar arquivos em disco quanto para monitorar processos em execução na memória.

2. VirusTotal: O Veredito Coletivo

O VirusTotal agrega mais de 60 motores de antivírus (AV).

- Dica de Segurança: Ao analisar algo confidencial, pesquise apenas o hash no VirusTotal. Nunca envie o arquivo original, pois isso pode alertar o atacante de que sua amostra está sendo investigada.

Capa: Traduzindo Código em Táticas de Ataque

O Capa (da Mandiant) é uma das ferramentas mais poderosas da FLARE VM. Ele não apenas lê o arquivo, mas "deduz" o que o malware é capaz de fazer, mapeando os resultados para frameworks globais como o MITRE ATT&CK.

Como o Capa organiza a análise:

- ATT&CK Tactic/Technique: Identifica o objetivo do invasor (ex: Evasão de Defesa, Descoberta de Sistema).
- MBC (Malware Behavior Catalog): Detalha comportamentos específicos do código (ex: Criptografia, Persistência).
- Capability: Lista as ações exatas que o malware pode realizar (ex: "deletar arquivo", "verificar se está em uma VM").

Exercícios de Laboratório: Capacidades de Malware (Amostra 4)

Após executar o comando `capa mal\4` no terminal, utilizei os resultados para responder:

- **Quantas correspondências para técnicas anti-execução de VM foram identificadas na amostra?**
  Resposta: 86

```
+-------------------------------------------------------+-------------------------------------------------------+
| CAPABILITY                                            | NAMESPACE                                             |
|-------------------------------------------------------+-------------------------------------------------------|
| check for PEB BeingDebugged flag                      | anti-analysis/anti-debugging/debugger-detection       |
| check for PEB NtGlobalFlag flag (2 matches)           | anti-analysis/anti-debugging/debugger-detection       |
| check for software breakpoints (4 matches)            | anti-analysis/anti-debugging/debugger-detection       |
| check for unmoving mouse cursor                       | anti-analysis/anti-vm/vm-detection                    |
| execute anti-VM instructions (86 matches)             | anti-analysis/anti-vm/vm-detection                    |
| contain obfuscated stackstrings (18 matches)          | anti-analysis/obfuscation/string/stackstring          |
| log keystrokes via polling (2 matches)                | collection/keylog                                     |
| check HTTP status code                                | communication/http/client                             |
| encode data using XOR (4 matches)                     | data-manipulation/encoding/xor                        |
| hash data using murmur3 (3 matches)                   | data-manipulation/hashing/murmur                      |
| authenticate HMAC                                     | data-manipulation/hmac                                |
| contains PDB path                                     | executable/pe/pdb                                     |
```

- **Em qual endereço está localizada a função que possui a capacidade de 'Check HTTP Status Code'?**
  (comando no powershell: capa -vv "mal\4" | Set-Content arquivo.txt)
  Resposta: 0x486921

```
check HTTP status code
namespace  communication/http/client
author     moritz.raabe@fireeye.com
scope      function
mbc        Communication::HTTP Communication::Read Header [
examples   54ac78733552a62d1d05ea4ba3fc604bb49fe000d7fc948d
function @ 0x486921
  and:
    or:
      number: 0x13 = HTTP_QUERY_STATUS_CODE @ 0x486E5A, 0x4
    subscope:
      and:
```

```
d-----        9/13/2022   7:40 AM                mal
d-----       11/15/2025   7:47 AM                PS_Transcripts
d-----        9/10/2022  11:12 PM                ssdeep-2.14.1
-a----        12/7/2021   3:26 PM            730 fakenet_logs.lnk
-a----         9/4/2022  12:01 AM           1328 FLARE.lnk
-a----        12/7/2021   3:26 PM           1579 README.txt

FLARE 4/3/2026 3:33:02 PM
PS C:\Users\Administrator\desktop > capa -vv "mal\4" | Set-Content arqu
ivo.txt
loading : 100%|
    | 485/485 [00:00<00:00, 2069.41     rules/s]
matching: 100%|
    | 7123/7123 [01:50<00:00, 64.28 functions/s]
FLARE 4/3/2026 3:36:21 PM
```

Bibliotecas e Importações: O Esqueleto da Ação

Um arquivo PE raramente trabalha sozinho. Para economizar espaço e complexidade, ele "pede emprestado" código do próprio Windows através de bibliotecas chamadas **DLLs** (Dynamic Link Libraries). Ao analisar quais DLLs um arquivo carrega, conseguimos prever quase com 100% de certeza o que ele é capaz de fazer.

No **PEstudio**, a aba *Libraries* mostra essa lista. Se um malware quer navegar na internet, ele precisa de bibliotecas de rede; se quer criptografar seus arquivos, precisará de bibliotecas de criptografia. O PEstudio mantém uma **"lista negra"** de bibliotecas que são frequentemente abusadas por malwares para realizar conexões externas ou injetar código.

Exercício de Fixação: Análise de DLLs (Amostra 4)

Abra o arquivo `Desktop\mal\4` no PEstudio e responda com base na aba *Libraries*:

- **Qual biblioteca está na lista negra e o que faz essa DLL?**
  - **Resposta:** `rpcrt4.dll`, Tempo de execução da Chamada de Procedimento Remoto (Remote Procedure Call Runtime)

library (9)	blacklist (1)	type (1)	functions (422)	description
kernel32.dll	-	implicit	153	Windows NT BASE API Client DLL
user32.dll	-	implicit	159	Multi-User Windows USER API Client DLL
gdi32.dll	-	implicit	79	GDI Client DLL
comdlg32.dll	-	implicit	3	Common Dialogs DLL
winspool.drv	-	implicit	3	Windows Spooler Driver
advapi32.dll	-	implicit	9	Advanced Windows 32 Base API
shell32.dll	-	implicit	6	Windows Shell Common Dll
comctl32.dll	-	implicit	9	Common Controls Library
rpcrt4.dll	x	implicit	1	Remote Procedure Call Runtime

Resumo Final: O Fluxo da Análise Estática Básica

Para encerrar o módulo, aqui está o checklist mental que você deve levar para o seu caderno:

1. **Segurança Primeiro:** Sempre utilize **Máquinas Virtuais** (FLARE VM / REMnux) e tire **Snapshots** antes de tocar em qualquer amostra.
2. **Identificação e Similaridade:** Use **Hashes (SHA-256)** para identificar o arquivo, **Imphash** para encontrar "parentes" pela estrutura de funções e **ssdeep** para medir a similaridade real do código.
3. **Busca por Pistas:** Extraia **Strings** para achar IPs, URLs e comandos. Se não houver nada, use o **FLOSS** para revelar o que está ofuscado.
4. **Mapeamento de Táticas:** Utilize o **Capa** para traduzir o código técnico em táticas do mundo real (Framework **MITRE ATT&CK**), identificando se o malware possui técnicas **Anti-VM**.
5. **Análise de Cabeçalho:** Verifique as **Importações** no PEstudio para entender quais "ferramentas" do Windows o arquivo está usando para executar sua missão maliciosa.